

NAME : JEEVA P
REG NO: 192412636
ITA0609

MACHINE LEARNING LAB MANUAL

Program 1: Find-S Algorithm to Find the Most Specific Hypothesis

Aim

To implement the Find-S algorithm in Python to find the most specific hypothesis that is consistent with the given set of positive training examples.

Algorithm

1. Start with the training dataset consisting of attribute values and a target concept (Yes/No).
2. Initialize the most specific hypothesis h as ['0','0','0','0','0','0'] (each '0' meaning no value seen yet).
3. For each training example, check whether the target output is 'Yes' (positive example).
4. If it is positive, then for every attribute: if the hypothesis value is '0', replace it with the attribute value from the example.
5. If the hypothesis value differs from the attribute value of the example, replace it with '?' (indicating the attribute can take any value).
6. Ignore all negative examples.
7. Repeat the process for all training examples.
8. Output the final hypothesis, which is the most specific hypothesis consistent with all positive examples.

Program

```
import numpy as np

data = [
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
]

hypothesis = ['0', '0', '0', '0', '0', '0']

for row in data:
    if row[-1] == 'Yes':
        for i in range(6):
            if hypothesis[i] == '0':
                hypothesis[i] = row[i]
            elif hypothesis[i] != row[i]:
                hypothesis[i] = '?'

print("Most Specific Hypothesis:")
print(hypothesis)
```

Input and Output

Input:

No user input required. The training dataset is defined within the program.

Output:

```
Most Specific Hypothesis:  
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

The Find-S algorithm was successfully implemented, and the most specific hypothesis consistent with the given positive training examples was obtained.

Program 2: Displaying Training Data with Positive and Negative Examples

Aim

To write a Python program to read a training dataset and separately display the complete data, the positive examples, and the negative examples.

Algorithm

1. Define the training dataset as a list of examples, each having attribute values and a target class label (Yes/No).
2. Display the entire training dataset row by row.
3. Traverse the dataset and check the target label of each row.
4. If the label is 'Yes', print the row under Positive Examples.
5. If the label is 'No', print the row under Negative Examples.
6. Repeat until all rows have been classified and displayed.
7. Stop.

Program

```
data = [  
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes'],  
    ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes'],  
    ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No'],  
    ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']  
]  
  
print("Training Data:")  
  
for row in data:  
    print(row)  
  
print("\nPositive Examples:")  
  
for row in data:  
    if row[-1] == 'Yes':  
        print(row)  
  
print("\nNegative Examples:")  
  
for row in data:  
    if row[-1] == 'No':  
        print(row)
```

Input and Output

Input:

No user input required. The training dataset is defined within the program.

Output:

```
Training Data:
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Positive Examples:
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Negative Examples:
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
```

Result

The program successfully read the training dataset and displayed the complete data along with the positive and negative examples separately.

Program 3: Decision Tree Classifier to Predict Result Based on Study and Attendance

Aim

To implement a Decision Tree Classifier using scikit-learn to predict a student's result based on study habit and attendance, using entropy as the splitting criterion.

Algorithm

1. Import DecisionTreeClassifier from sklearn.tree and numpy.
2. Prepare the feature matrix X (Study, Attendance) and target vector y (Result) from the training data.
3. Create a DecisionTreeClassifier object with criterion set to 'entropy'.
4. Train (fit) the model using the training data X and y.
5. Accept Study and Attendance values as input from the user.
6. Use the trained model to predict the result for the given input.
7. Display the predicted result.
8. Stop.

Program

```
from sklearn.tree import DecisionTreeClassifier
import numpy as np

X = np.array([
    [1, 1],
    [1, 0],
    [0, 1],
    [0, 0],
    [1, 1],
    [0, 0]
])

y = np.array([1, 1, 0, 0, 1, 0])

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

study = int(input("Enter Study (1/0): "))
attendance = int(input("Enter Attendance (1/0): "))

print("Predicted Result:", model.predict([[study, attendance]])[0])
```

Input and Output

Input:

```
Enter Study (1/0): 1
Enter Attendance (1/0): 1
```

Output:

```
Predicted Result: 1
```

Result

The Decision Tree Classifier was successfully trained and used to predict the result based on study and attendance values entered by the user.

Program 4: Multi-Layer Perceptron (MLP) Classifier to Simulate a Logic Gate

Aim

To implement a Multi-Layer Perceptron Classifier using scikit-learn to learn and predict the output of a logic gate (OR gate) given two binary inputs.

Algorithm

1. Import MLPClassifier from sklearn.neural_network and numpy.
2. Prepare the input feature matrix X containing all combinations of two binary inputs (0,0), (0,1), (1,0), (1,1).
3. Prepare the target output vector y corresponding to the OR gate logic.
4. Create an MLPClassifier object with one hidden layer of 5 neurons and a maximum of 2000 iterations.
5. Train (fit) the model using X and y.
6. Accept two binary values (0/1) as input from the user.
7. Use the trained model to predict the output for the given input pair.
8. Display the predicted output.
9. Stop.

Program

```
from sklearn.neural_network import MLPClassifier
import numpy as np

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 1, 1, 1])

model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=2000, random_state=1)
model.fit(X, y)

a = int(input("Enter first input (0/1): "))
b = int(input("Enter second input (0/1): "))

print("Predicted Output:", model.predict([[a, b]])[0])
```

Input and Output

Input:

```
Enter first input (0/1): 1
Enter second input (0/1): 0
```

Output:

```
Predicted Output: 1
```


Result

The MLP Classifier was successfully trained on the logic gate truth table and correctly predicted the output for the given binary inputs.

Program 5: K-Nearest Neighbors (KNN) Classifier to Classify Income Level

Aim

To implement a K-Nearest Neighbors Classifier using scikit-learn to classify a person's income category (Low/High) based on age and income values.

Algorithm

1. Import KNeighborsClassifier from sklearn.neighbors and numpy.
2. Prepare the feature matrix X (Age, Income) and the target class vector y (Low/High).
3. Create a KNeighborsClassifier object with the number of neighbors (k) set to 3.
4. Train (fit) the model using the training data X and y.
5. Accept Age and Income values as input from the user.
6. Use the trained model to predict the class for the given input.
7. Display the predicted class.
8. Stop.

Program

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

X = np.array([
    [1, 20],
    [2, 21],
    [3, 22],
    [7, 30],
    [8, 31],
    [9, 32]
])

y = np.array(['Low', 'Low', 'Low', 'High', 'High', 'High'])

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

age = float(input("Enter Age: "))
income = float(input("Enter Income: "))

print("Predicted Class:", model.predict([[age, income]])[0])
```

Input and Output

Input:

```
Enter Age: 5
Enter Income: 25
```

Output:

```
Predicted Class: Low
```

Result

The KNN Classifier was successfully trained and used to predict the income class based on the age and income entered by the user.

Program 6: Naive Bayes Classifier with Accuracy and Confusion Matrix

Aim

To implement a Gaussian Naive Bayes Classifier using scikit-learn to classify a person's category based on age and student status, and to evaluate the model using accuracy and a confusion matrix.

Algorithm

1. Import GaussianNB from sklearn.naive_bayes and accuracy_score, confusion_matrix from sklearn.metrics.
2. Prepare the feature matrix X (Age, Student status) and the target class vector y.
3. Create a GaussianNB object and train (fit) it using X and y.
4. Accept Age and Student status (1/0) as input from the user.
5. Use the trained model to predict the class for the given input and display it.
6. Compute the model's predictions on the training data X.
7. Calculate and display the accuracy score by comparing actual y with predicted values.
8. Calculate and display the confusion matrix.
9. Stop.

Program

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

X = np.array([
    [20, 1],
    [25, 1],
    [30, 0],
    [35, 0],
    [40, 0],
    [22, 1]
])

y = np.array([1, 1, 0, 0, 0, 1])

model = GaussianNB()
model.fit(X, y)

age = float(input("Enter Age: "))
student = int(input("Are you a Student? (1/0): "))

prediction = model.predict([[age, student]])

print("Predicted Class:", prediction[0])
print("Accuracy:", accuracy_score(y, model.predict(X)))
print("Confusion Matrix:")
print(confusion_matrix(y, model.predict(X)))
```

Input and Output

Input:

Enter Age: 23

Are you a Student? (1/0): 1

Output:

```
Predicted Class: 1
Accuracy: 1.0
Confusion Matrix:
[[3 0]
 [0 3]]
```

Result

The Naive Bayes Classifier was successfully trained and evaluated, achieving an accuracy of 1.0 on the training data, and the predicted class along with the confusion matrix was displayed.

Program 7: Logistic Regression to Predict Pass/Fail Result

Aim

To implement Logistic Regression using scikit-learn to predict whether a student passes or fails based on the number of study hours.

Algorithm

1. Import LogisticRegression from sklearn.linear_model and numpy.
2. Prepare the feature matrix X (Study Hours) and the binary target vector y (Pass=1/Fail=0).
3. Create a LogisticRegression object and train (fit) it using X and y.
4. Accept the number of study hours as input from the user.
5. Use the trained model to predict the result for the given number of hours.
6. Display the predicted result.
7. Stop.

Program

```
from sklearn.linear_model import LogisticRegression
import numpy as np

X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(X, y)

hours = float(input("Enter Study Hours: "))

print("Predicted Result:", model.predict([[hours]])[0])
```

Input and Output

Input:

Enter Study Hours: 5

Output:

Predicted Result: 1

Result

The Logistic Regression model was successfully trained and used to predict the pass/fail result based on the study hours entered by the user.

Program 8: Linear Regression to Predict Marks Based on Study Hours

Aim

To implement Linear Regression using scikit-learn to predict a student's marks based on the number of hours studied.

Algorithm

1. Import LinearRegression from sklearn.linear_model and numpy.
2. Prepare the feature matrix X (Study Hours) and the target vector y (Marks).
3. Create a LinearRegression object and train (fit) it using X and y.
4. Accept the number of study hours as input from the user.
5. Use the trained model to predict the marks for the given number of hours.
6. Display the predicted marks.
7. Stop.

Program

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1],[2],[3],[4],[5],[6]])
y = np.array([35,45,55,65,75,85])

model = LinearRegression()
model.fit(X, y)

hours = float(input("Enter Study Hours: "))

print("Predicted Marks:", model.predict([[hours]])[0])
```

Input and Output

Input:

Enter Study Hours: 5

Output:

Predicted Marks: 75.0

Result

The Linear Regression model was successfully trained and used to predict marks based on the study hours entered by the user.

Program 9: Linear vs Polynomial Regression to Predict Value Based on Temperature

Aim

To implement and compare Linear Regression and Polynomial Regression using scikit-learn to predict a value based on the given temperature.

Algorithm

1. Import LinearRegression from sklearn.linear_model and PolynomialFeatures from sklearn.preprocessing.
2. Prepare the feature matrix X (Temperature) and the target vector y.
3. Create and train a LinearRegression model using X and y.
4. Transform X into polynomial features of degree 2 using PolynomialFeatures.
5. Create and train a second LinearRegression model using the transformed polynomial features and y.
6. Accept the temperature value as input from the user.
7. Predict the output using the linear model directly on the input.
8. Transform the input using the same polynomial transformer and predict the output using the polynomial model.
9. Display both the linear regression prediction and the polynomial regression prediction.
10. Stop.

Program

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

X = np.array([[10], [15], [20], [25], [30], [35]])
y = np.array([20, 35, 55, 80, 120, 170])

# Linear Regression
linear_model = LinearRegression()
linear_model.fit(X, y)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)

temperature = float(input("Enter Temperature: "))

linear_prediction = linear_model.predict([[temperature]])[0]

poly_prediction = poly_model.predict(
    poly.transform([[temperature]])
)[0]

print("Linear Regression Prediction:", linear_prediction)
print("Polynomial Regression Prediction:", poly_prediction)
```


Input and Output

Input:

Enter Temperature: 22

Output:

Linear Regression Prediction: 77.05714285714285
Polynomial Regression Prediction: 63.82142857142862

Result

Both Linear Regression and Polynomial Regression models were successfully trained, and their predictions for the given temperature were compared and displayed.

Program 10: Gaussian Mixture Model for Clustering

Aim

To implement a Gaussian Mixture Model (GMM) using scikit-learn to cluster a given set of values into groups and predict the cluster of a new value.

Algorithm

1. Import GaussianMixture from sklearn.mixture and numpy.
2. Prepare the feature matrix X containing the data points to be clustered.
3. Create a GaussianMixture object with the number of components (clusters) set to 2.
4. Train (fit) the model using the data X.
5. Accept a value as input from the user.
6. Use the trained model to predict the cluster to which the input value belongs.
7. Display the predicted cluster.
8. Stop.

Program

```
from sklearn.mixture import GaussianMixture
import numpy as np

X = np.array([
    [1], [2], [3], [8], [9], [10]
])

model = GaussianMixture(n_components=2, random_state=1)
model.fit(X)

value = float(input("Enter a value: "))

print("Predicted Cluster:", model.predict([[value]])[0])
```

Input and Output

Input:

Enter a value: 5

Output:

Predicted Cluster: 0

Result

The Gaussian Mixture Model successfully clustered the given data into two groups and correctly predicted the cluster for the input value.

Program 11: Decision Tree Classifier to Predict Credit Score

Aim

To implement a Decision Tree Classifier using scikit-learn to predict a person's credit score category (Poor/Average/Good) based on age and annual income.

Algorithm

1. Import DecisionTreeClassifier from sklearn.tree and numpy.
2. Prepare the feature matrix X (Age, Income) and the target class vector y (Poor/Average/Good).
3. Create a DecisionTreeClassifier object and train (fit) it using X and y.
4. Accept Age and Annual Income as input from the user.
5. Use the trained model to predict the credit score category for the given input.
6. Display the predicted credit score.
7. Stop.

Program

```
from sklearn.tree import DecisionTreeClassifier
import numpy as np

X = np.array([
    [20, 30000],
    [25, 40000],
    [30, 50000],
    [35, 60000],
    [40, 70000],
    [45, 80000]
])

y = np.array(['Poor', 'Poor', 'Average', 'Average', 'Good', 'Good'])

model = DecisionTreeClassifier()
model.fit(X, y)

age = float(input("Enter Age: "))
income = float(input("Enter Annual Income: "))

print("Credit Score:", model.predict([[age, income]])[0])
```

Input and Output

Input:

```
Enter Age: 32
Enter Annual Income: 55000
```

Output:

```
Credit Score: Average
```

Result

The Decision Tree Classifier was successfully trained and used to predict the credit score category based on the age and income entered by the user.

Program 12: KNN Classifier to Predict Iris Flower Species

Aim

To implement a K-Nearest Neighbors Classifier using scikit-learn on the built-in Iris dataset to predict the species of a flower based on its sepal and petal measurements.

Algorithm

1. Import load_iris from sklearn.datasets and KNeighborsClassifier from sklearn.neighbors.
2. Load the Iris dataset and separate it into feature matrix X and target vector y.
3. Create a KNeighborsClassifier object with the number of neighbors (k) set to 3.
4. Train (fit) the model using X and y.
5. Accept sepal length, sepal width, petal length, and petal width as input from the user.
6. Use the trained model to predict the flower species for the given measurements.
7. Map the predicted numeric class to its corresponding flower name using target_names.
8. Display the predicted flower species.
9. Stop.

Program

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

length = float(input("Enter Sepal Length: "))
width = float(input("Enter Sepal Width: "))
plength = float(input("Enter Petal Length: "))
pwidth = float(input("Enter Petal Width: "))

prediction = model.predict([[length, width, plength, pwidth]])

print("Predicted Flower:", iris.target_names[prediction[0]])
```

Input and Output

Input:

```
Enter Sepal Length: 5.1
Enter Sepal Width: 3.5
Enter Petal Length: 1.4
Enter Petal Width: 0.2
```

Output:

```
Predicted Flower: setosa
```

Result

The KNN Classifier was successfully trained on the Iris dataset and correctly predicted the flower species based on the measurements entered by the user.

Program 13: Linear Regression to Predict Car Price

Aim

To implement Linear Regression using scikit-learn to predict the price of a used car based on its age and kilometers driven.

Algorithm

1. Import LinearRegression from sklearn.linear_model and numpy.
2. Prepare the feature matrix X (Car Age, Kilometers Driven) and the target vector y (Price).
3. Create a LinearRegression object and train (fit) it using X and y.
4. Accept the car's age and kilometers driven as input from the user.
5. Use the trained model to predict the car price for the given input.
6. Display the predicted car price.
7. Stop.

Program

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([
    [2, 50000],
    [3, 45000],
    [4, 40000],
    [5, 35000],
    [6, 30000],
    [7, 25000]
])

y = np.array([800000, 700000, 600000, 500000, 400000, 300000])

model = LinearRegression()
model.fit(X, y)

age = float(input("Enter Car Age: "))
km = float(input("Enter Kilometers Driven: "))

print("Predicted Car Price:", model.predict([[age, km]])[0])
```

Input and Output

Input:

```
Enter Car Age: 5
Enter Kilometers Driven: 40000
```

Output:

```
Predicted Car Price: 599999.99600000002
```

Result

The Linear Regression model was successfully trained and used to predict the car price based on the age and kilometers driven entered by the user.

Program 14: Linear Regression to Predict House Price

Aim

To implement Linear Regression using scikit-learn to predict the price of a house based on its area and number of rooms.

Algorithm

1. Import LinearRegression from sklearn.linear_model and numpy.
2. Prepare the feature matrix X (Area, Number of Rooms) and the target vector y (Price).
3. Create a LinearRegression object and train (fit) it using X and y.
4. Accept the house area and number of rooms as input from the user.
5. Use the trained model to predict the house price for the given input.
6. Display the predicted house price.
7. Stop.

Program

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([
    [500, 1],
    [700, 2],
    [900, 2],
    [1100, 3],
    [1300, 3],
    [1500, 4]
])

y = np.array([2000000, 2800000, 3500000, 4500000, 5200000, 6000000])

model = LinearRegression()
model.fit(X, y)

area = float(input("Enter House Area in sq.ft: "))
rooms = float(input("Enter Number of Rooms: "))

print("Predicted House Price:", model.predict([[area, rooms]])[0])
```

Input and Output

Input:

```
Enter House Area in sq.ft: 1000
Enter Number of Rooms: 2
```

Output:

```
Predicted House Price: 3933333.333333333
```

Result

The Linear Regression model was successfully trained and used to predict the house price based on the area and number of rooms entered by the user.

Program 15: Naive Bayes Classifier to Predict Iris Flower Species

Aim

To implement a Gaussian Naive Bayes Classifier using scikit-learn on the built-in Iris dataset to predict the species of a flower based on its measurements.

Algorithm

1. Import load_iris from sklearn.datasets and GaussianNB from sklearn.naive_bayes.
2. Load the Iris dataset and separate it into feature matrix X and target vector y.
3. Create a GaussianNB object and train (fit) it using X and y.
4. Accept sepal length, sepal width, petal length, and petal width as input from the user.
5. Use the trained model to predict the flower species for the given measurements.
6. Map the predicted numeric class to its corresponding flower name using target_names.
7. Display the predicted flower species.
8. Stop.

Program

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

iris = load_iris()

X = iris.data
y = iris.target

model = GaussianNB()
model.fit(X, y)

sl = float(input("Enter Sepal Length: "))
sw = float(input("Enter Sepal Width: "))
pl = float(input("Enter Petal Length: "))
pw = float(input("Enter Petal Width: "))

prediction = model.predict([[sl, sw, pl, pw]])

print("Predicted Flower:", iris.target_names[prediction[0]])
```

Input and Output

Input:

```
Enter Sepal Length: 6.0
Enter Sepal Width: 2.9
Enter Petal Length: 4.5
Enter Petal Width: 1.5
```

Output:

```
Predicted Flower: versicolor
```

Result

The Naive Bayes Classifier was successfully trained on the Iris dataset and correctly predicted the flower species based on the measurements entered by the user.

Program 16: Comparison of Classification Algorithms on the Breast Cancer Dataset

Aim

To implement and compare the performance of Logistic Regression, Decision Tree, and K-Nearest Neighbors classifiers on the built-in Breast Cancer dataset using accuracy as the evaluation metric.

Algorithm

1. Import `load_breast_cancer` from `sklearn.datasets`, `train_test_split` from `sklearn.model_selection`, and the required classifiers along with `accuracy_score`.
2. Load the Breast Cancer dataset and separate it into feature matrix `X` and target vector `y`.
3. Split the dataset into training and testing sets (80% train, 20% test) using `train_test_split`.
4. Create a list containing `LogisticRegression`, `DecisionTreeClassifier`, and `KNeighborsClassifier` model objects.
5. For each model in the list, train (fit) it on the training data.
6. Use each trained model to predict the labels for the test data.
7. Calculate the accuracy score of each model by comparing predicted and actual test labels.
8. Display the name of each model along with its accuracy.
9. Stop.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = [
    LogisticRegression(max_iter=5000),
    DecisionTreeClassifier(),
    KNeighborsClassifier()
]

for model in models:
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    print(type(model).__name__, "Accuracy:",
          accuracy_score(y_test, prediction))
```

Input and Output

Input:

No user input required. The Breast Cancer dataset is loaded automatically within the program.

Output:

LogisticRegression Accuracy: 0.956140350877193
DecisionTreeClassifier Accuracy: 0.9298245614035088
KNeighborsClassifier Accuracy: 0.956140350877193

Result

Three classification algorithms were successfully trained and evaluated on the Breast Cancer dataset, and their accuracies were compared, with Logistic Regression and KNN achieving the highest accuracy among the three.

Program 17: Decision Tree Classifier to Predict Mobile Price Range

Aim

To implement a Decision Tree Classifier using scikit-learn to predict the price range (Low/Medium/High) of a mobile phone based on its RAM and storage capacity.

Algorithm

1. Import DecisionTreeClassifier from sklearn.tree and numpy.
2. Prepare the feature matrix X (RAM, Storage) and the target class vector y (Low/Medium/High).
3. Create a DecisionTreeClassifier object and train (fit) it using X and y.
4. Accept RAM (in GB) and Storage (in GB) as input from the user.
5. Use the trained model to predict the mobile price range for the given input.
6. Display the predicted mobile price range.
7. Stop.

Program

```
from sklearn.tree import DecisionTreeClassifier
import numpy as np

X = np.array([
    [2, 32],
    [3, 64],
    [4, 128],
    [6, 128],
    [8, 256],
    [12, 256]
])

y = np.array([
    'Low',
    'Low',
    'Medium',
    'Medium',
    'High',
    'High'
])

model = DecisionTreeClassifier()
model.fit(X, y)

ram = float(input("Enter RAM in GB: "))
storage = float(input("Enter Storage in GB: "))

print("Predicted Mobile Price Range:",
      model.predict([[ram, storage]])[0])
```

Input and Output

Input:

```
Enter RAM in GB: 4
Enter Storage in GB: 128
```

Output:

Predicted Mobile Price Range: Medium

Result

The Decision Tree Classifier was successfully trained and used to predict the mobile price range based on the RAM and storage entered by the user.

Program 18: Perceptron Classifier for Binary Classification on Iris Dataset

Aim

To implement a Perceptron Classifier using scikit-learn on the Iris dataset to perform binary classification based on sepal length and sepal width.

Algorithm

1. Import load_iris from sklearn.datasets and Perceptron from sklearn.linear_model.
2. Load the Iris dataset and select the first 100 samples (two classes) and the first two features (sepal length, sepal width).
3. Create a Perceptron object and train (fit) it using the selected feature matrix X and target vector y.
4. Accept sepal length and sepal width as input from the user.
5. Use the trained model to predict the class for the given input.
6. Display the predicted class.
7. Stop.

Program

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()

X = iris.data[:100, :2]
y = iris.target[:100]

model = Perceptron()
model.fit(X, y)

sl = float(input("Enter Sepal Length: "))
sw = float(input("Enter Sepal Width: "))

prediction = model.predict([[sl, sw]])

print("Predicted Class:", prediction[0])
```

Input and Output

Input:

```
Enter Sepal Length: 5.0
Enter Sepal Width: 3.5
```

Output:

```
Predicted Class: 0
```

Result

The Perceptron Classifier was successfully trained on the Iris dataset subset and correctly predicted the class based on the sepal measurements entered by the user.

Program 19: Naive Bayes Classifier to Predict Loan Approval

Aim

To implement a Gaussian Naive Bayes Classifier using scikit-learn to predict whether a loan will be approved based on a person's age and annual income.

Algorithm

1. Import GaussianNB from sklearn.naive_bayes and numpy.
2. Prepare the feature matrix X (Age, Annual Income) and the target class vector y (Yes/No).
3. Create a GaussianNB object and train (fit) it using X and y.
4. Accept Age and Annual Income as input from the user.
5. Use the trained model to predict the loan approval status for the given input.
6. Display the predicted loan approval result.
7. Stop.

Program

```
from sklearn.naive_bayes import GaussianNB
import numpy as np

X = np.array([
    [25, 30000],
    [30, 40000],
    [35, 50000],
    [40, 60000],
    [45, 70000],
    [50, 80000]
])

y = np.array([
    'No',
    'No',
    'Yes',
    'Yes',
    'Yes',
    'Yes'
])

model = GaussianNB()
model.fit(X, y)

age = float(input("Enter Age: "))
income = float(input("Enter Annual Income: "))

print("Loan Prediction:", model.predict([[age, income]])[0])
```

Input and Output

Input:

Enter Age: 38

Enter Annual Income: 55000

Output:

Loan Prediction: Yes

Result

The Naive Bayes Classifier was successfully trained and used to predict the loan approval status based on the age and income entered by the user.

Program 20: Linear Regression to Predict Future Sales

Aim

To implement Linear Regression using scikit-learn to predict future sales based on the month number.

Algorithm

1. Import LinearRegression from sklearn.linear_model and numpy.
2. Prepare the feature matrix X (Month Number) and the target vector y (Sales).
3. Create a LinearRegression object and train (fit) it using X and y.
4. Accept a future month number as input from the user.
5. Use the trained model to predict the sales for the given month.
6. Display the predicted sales.
7. Stop.

Program

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([
    [1], [2], [3], [4], [5], [6]
])

y = np.array([
    120, 150, 180, 210, 250, 280
])

model = LinearRegression()
model.fit(X, y)

month = float(input("Enter Future Month Number: "))

print("Predicted Sales:", model.predict([[month]])[0])
```

Input and Output

Input:

Enter Future Month Number: 7

Output:

Predicted Sales: 311.33333333333337

Result

The Linear Regression model was successfully trained and used to predict future sales based on the month number entered by the user.